



3

12

1800sp

نظري

كلية الهندسة المعلوماتية

السنة الرابعة

Heuristic Search

د. باسل الخطيب



RB Informatcs; 19/10/2025

محتوى مجاني غير مخصص للبيع التجاري

خوارزميات البحث الذكية

مقدمة:

تحدثنا في المحاضرة السابقة عن خوارزميات الـ uniformed search أو الـ blind search وهي الـ UCS,DFS,BFS وقلنا إن سبب تسميتها يعود إلى كونها لا تملك أي معلومة عن الاتجاه الأفضل نحو الهدف اليوم سنتعرف على خوارزمية الـ informed search (heuristic search) التي بنوا عليها الـ Hill climbing والـ A star.

■ ذكرنا سابقاً أن خوارزميات البحث العمياء تقوم بتجريب كل الاحتمالات بالتساوي مما يجعلها بطيئة جداً في المسائل الواقعية.

أما البحث بالـ heuristic فهو مثل وجود بوصلة أو مؤشر يقول هذا الطريق أقرب للمخرج فتقل عدد المحاولات سنوضح الفكرة أدناه

■ في الكثير من مسائل البحث نستطيع ملاحظة الحالة الأقرب للهدف

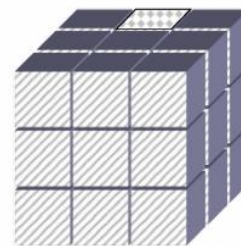
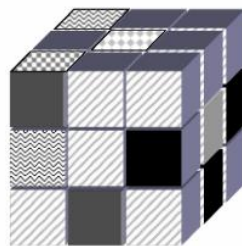
مثال 1:

7	8	4
3	5	1
6	2	

1	2	3
4	5	6
7		8

نلاحظ أن الشكل على اليمين هو الأقرب للهدف goal state حيث يبعد عنها خطوة واحدة وهي تحريك المربع رقم 8

مثال 2:



نلاحظ أيضاً أن الشكل على اليمين هو الأقرب للحالة الهدف.



- في الأمثلة السابقة استطعنا تحديد فيما إذا كانت اللعبة اقتربت من الحل أم لا وذلك بالنظر إلى شكل الـ state الحالة الراهنة وشكل الـ state هو معلومة تفيد في ابتكار خوارزميات جديدة مختلفة عن التي رأيناها سابقاً.

Heuristic Search

الاستدلال Heuristic:

ببساطة هو دالة تأخذ حالة كدخل وترجع رقم كخرج. هذا الرقم هو **تقدير** لمدى بعد الحالة عن الهدف بعبارة أخرى يوضح لنا الاستدلال المسافة **تقريباً** بين الحالة الحالية وحالة الهدف مثلاً رقم أصغر = أقرب للهدف.

خواص أساسية للاستدلال:

نلاحظ أننا قلنا **تقريباً** توضح لنا الخوارزمية المسافة من الحالة الراهنة إلى الحالة الهدف فهي قد تخطئ إما تقلل **underestimate** المسافة فتعطي قيمة أقل من القيمة الحقيقية. أو تبالغ **overestimate** فتعطي قيمة أكبر من القيمة الحقيقية للوصول للهدف.

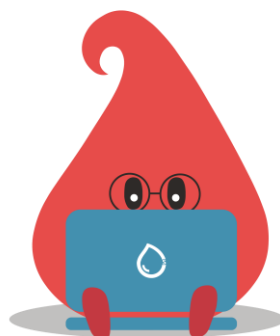
نتوصل إلى وجود نوعين من الاستدلال:

1. underestimate

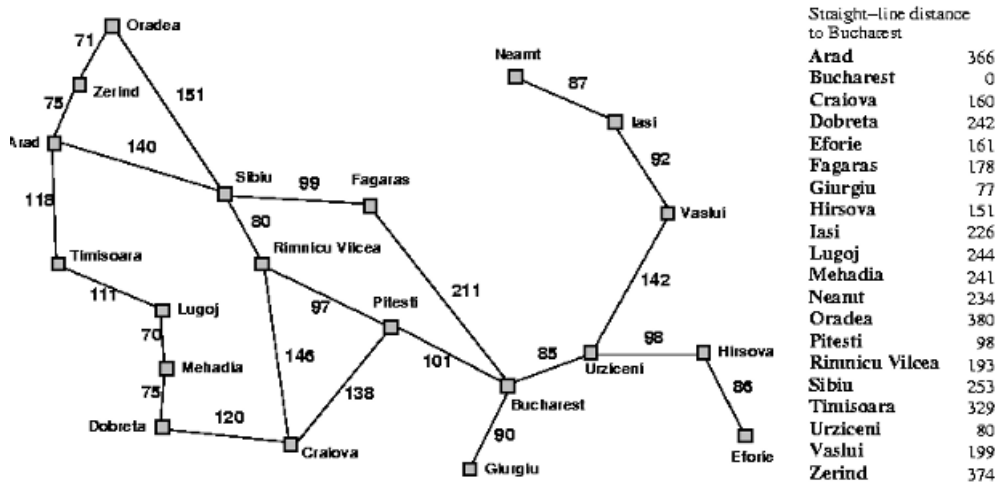
وهو النوع المرغوب وسمي أيضاً بالـ **admissible** المقبول أو الصالح. فالخوارزمية هنا لا تقدر أبداً قيمة أكبر من القيمة الحقيقية أي دوماً يكون التقدير هو قيمة أقل أو تساوي القيمة الحقيقية وهو النوع المرغوب لأنه يضمن ألا يفوت الحل الأمثل.

2. overestimate

وهو نوع غير مرغوب لأنه يقدر قيمة أكبر من القيمة الحقيقية فقد تظن الخوارزمية أن بعض المسارات سيئة جداً وتتجاهلها وقد تفوت الحل الأمثل.



بالعودة لمسألة Romania path:



عند البحث عن أقصر طريق من مدينة Arad إلى مدينة Bucharest يمكننا اعتبار الخط المستقيم straight-line بين المدينتين هو الطريق الأقصر حيث لا يمكن أن تكون المسافة الحقيقية بين المدينتين أقصر من المسافة المستقيمة بينهما وهذا ما نسميه بمسافة خط النظر وهي الأرقام التي على يمين الخريطة.

ملاحظة:

هنا اعتبرنا الـ straight-line هو heuristic من النوع underestimate هنا الـ heuristic محسوب ومعطى ضمن المسألة وهو الـ straight-line to Bucharest المسافة المستقيمة إلى الهدف مثلاً: $h(\text{Arad})=366$ يعني المسافة المستقيمة من Arad إلى Bucharest هي 366

فائدة أربسيزية:

Greedy Algorithms هي الخوارزميات التي تختار في كل خطوة الحل الأمثل بغض النظر إذا كان هذا الحل مفيد للخطوات القادمة أم لا وقد يكون هذا الحل صحيح ولكن ليس دائماً



من اعتاد أن يوزع الورد فسابقى شيء من العطر بيده
ما دمت تفعل الخير سيصلك أثره..

خوارزمية ال Hill Climbing Search

تعتمد على مبدأ الخوارزميات الشرهة Greedy لإيجاد الحل.

كيف تعمل الخوارزمية؟

عند كل خطوة ننظر لجيران الحالة الحالية أو الأبناء (الحالات الممكن الوصول إليها بخطوة واحدة) وتنقل إلى الجار الذي له أصغر قيمة $h(n)$ الأقرب للهدف حسب التقدير.
 $h(n)$ هو تقدير التكلفة من العقدة n إلى الهدف وهو الدالة الاستدلالية التي تحدثنا عنها في الفقرة السابقة .

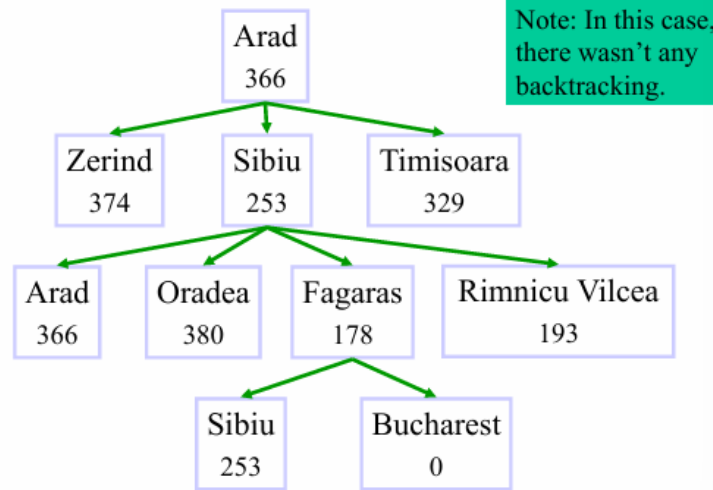
المقارنة مع ال UCS خوارزمية البحث بالتكلفة الواحدة:

UCS ينظر إلى تكلفة المسار الفعلية $g(n)$ من عقدة البداية أما ال Hill Climbing ينظر إلى قيمة ال $h(n)$ أي المسار الذي يبدو أقرب الآن.

عيوب الخوارزمية :

عندما تنتقل إلى العقدة ذات أقل قيمة $h(n)$ فإن الحل قد يكون صحيح أو أمثلي لكن ليس بالضرورة وقد يتسبب هذا الانتقال في الدخول ضمن حلقة loop أو طريق مسدود.
 مثلاً لو أننا نريد تسلق تل قد نصل للقمة لكن هذه القمة ليست القمة الأعلى في التل.

بالعودة للمثال السابق:



- هنا تم حساب straight-line distance ال $h(n)$ لكل مدينة إلى Bucharest نبدأ من Arad ثم نختار الابن صاحب ال h الأصغر وهكذا فيصبح الحل حسب الخوارزمية Hill Climbing :
 Arad => Sibiu => Fagaras => Bucharest
- هذا الحل قد يكون أمثلي وقد لا يكون
- نلاحظ هنا أنه لم يحصل backtracking أي الخوارزمية سارت بشكل متزايد نحو الهدف.

■ خصائص خوارزمية ال Hill Climbing :

1. الاكتمال completeness:

تفشل في فضاء الحالات ذو العمق اللامتناهي أو في الحلقات لكن في الفضاءات المنتهية ومع كشف الحالات المتكررة تصبح complete.

2. الأمثلية optimality :

لا لأن المؤشر هو حالي أي المسار الأمثل قد يكون بالمرور بحالة تبدو أبعد حالياً.

3. التعقيد الزمني:

في حال كانت منتهية فهو $O(bm)$ لأننا نقوم بتوليد b عقدة في كل مستوى ولدينا m مستوى.

4. تعقيد المساحة:

هو $O(b+m)$ في حال تم حفظ مسار الحل بالإضافة إلى أولاد العقدة الحالية حيث b هو عامل التفرع و m هو العمق الأعظمي.

ملاحظة:

قد تكون الخوارزمية غير منتهية ولا تتمكن من إيجاد الحل عندها تعقيدها يكون ∞ .

مثال 2:

heuristics for 8-puzzle

Goal State

1	2	3
4	5	6
7	8	

Current State

1	2	3
4	5	6
7		8

❖ ننظر إلى كل رقم إذا لم يكن في مكانه الصحيح نعدّه واحد مجموع هذه الأعداد هو $h(n)$

انتبه ! نحن لا نعد المربع الفارغ blank

❖ الـ $h(n)$ يخبرنا أننا قد نحتاج إلى خطوة واحدة على الأقل للوصول إلى الحل.

$$h(\text{current state})=1$$

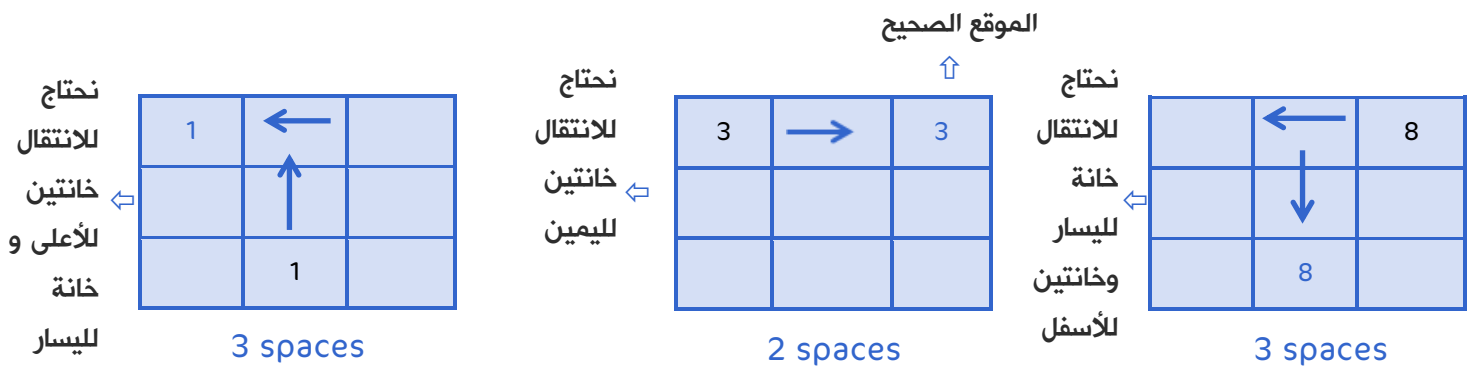
Goal State

1	2	3
4	5	6
7	8	

Current State

3	2	8
4	5	6
7	1	

أولاً نحدد المربعات التي مواقعها خاطئة نجد أنها 1,3,8

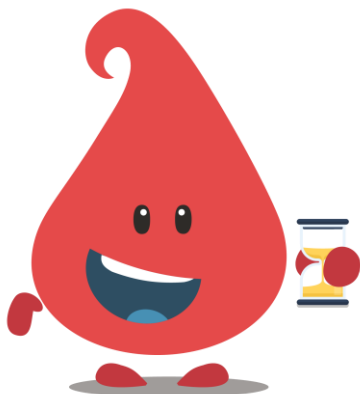


باستخدام مسافة مانهاتن نحسب المسافة الأفقية + العمودية من كل مربع خاطئ إلى موقعه الصحيح

نجمع كل هذه المسافات لتظهر لدينا قيمة الـ $h(n)$

وهي هنا $h(\text{current state}) = 3 + 3 + 2 = 8$ أي للوصول للحل من الحالة الحالية نحتاج على الأقل 8 حركات.

باستخدام مسافة مانهاتن نحصل على $h(n)$ أكثر دقة من عدد البلاطات الـ misplaced في المثال السابق.



سنكسب رهان الحياة يوماً ...
ما كان جهدنا على أحلامنا عبثاً



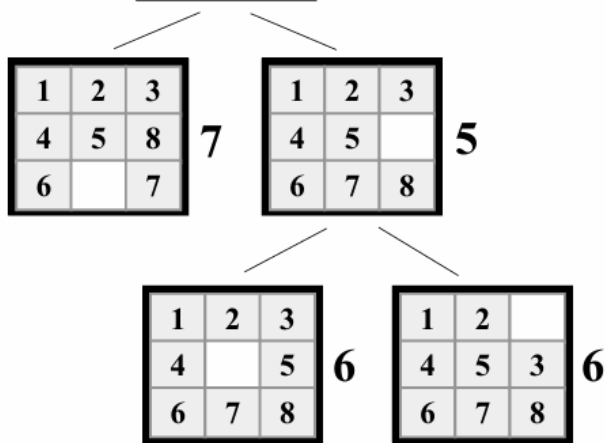
1	2	3
4	5	8
6	7	

6

$h(n)$

الـ heuristic سريعة لكن ليست مضمونة كما لديها عدة عيوب مثلاً قد نصل إلى حالة قد يكون كل الأبناء هم أسوأ خيار هذا ما نسميه local minima حيث لا توجد خطوة تقلل الـ h مع أن الحل قد يكون موجود لكن عبر مسار يزيد h مؤقتاً لكن تنخفض لاحقاً .

في هذه الحالة الخوارزمية إما تتوقف أو تدور في loop هذا ما يدفعنا إلى البحث عن خوارزميات توازن بين الـ h وتكلفة المسار الفعلي و



لقد مر معنا خوارزميتين مثيرتين للاهتمام:

1. Uniform cost search:

- تحسب الكلفة الفعلية للوصول لكل حالة.
- دائماً تعطي الحل الأفضل optimal
- دائماً تجد الحل إذا كان موجود complete
- بطيئة لأنها تجرب كل الحالات الممكنة.

2. Hill Climbing (Greedy Search):

- تحسب تقديرياً قرب الحالة من الهدف وليس الكلفة الفعلية.
- سريعة جداً لأنها تختار الحالة التي تبدو الأقرب للهدف.
- لكنها ليست مضمونة ولا تعطي الحل الأمثل بالضرورة.

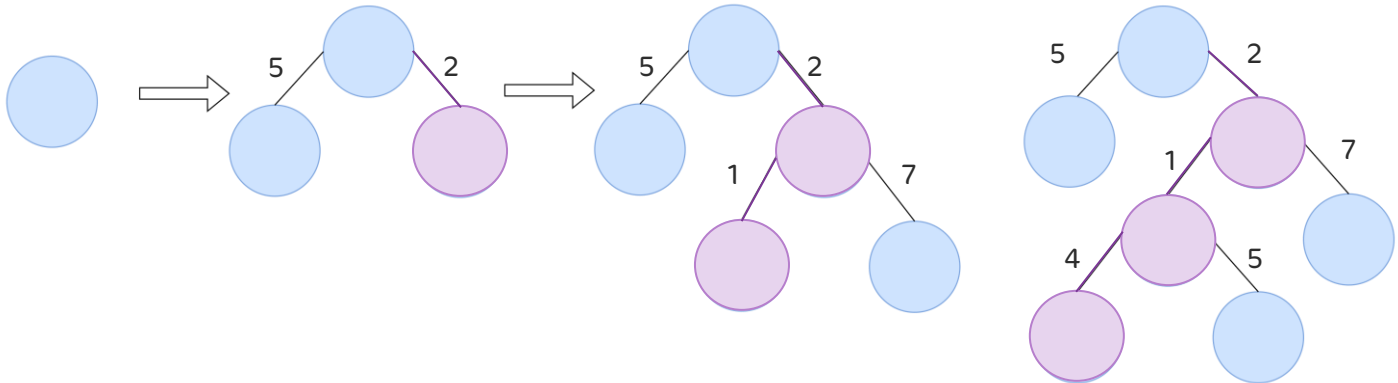
السؤال هنا هل يمكن دمج الخوارزميتين؟

الجواب نعم حيث نأخذ من Uniform Cost الضمان والحل الأمثل والسرعة والذكاء من Hill Climbing لتظهر لدينا خوارزمية A star (A*) لنوضح الخوارزمية بالأمثلة التالية:



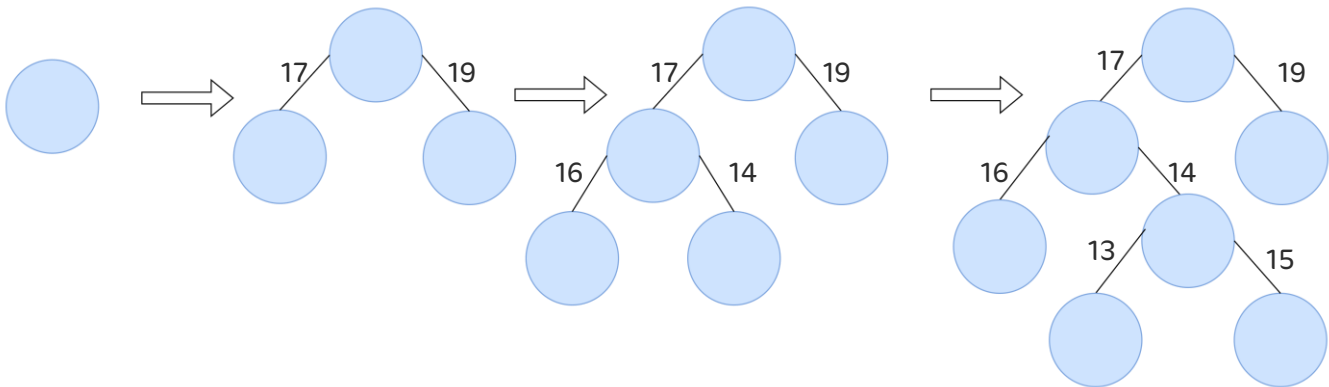
Uniform cost

المبدأ: أختار أقل كلفة لـ $g(n)$ أي أختار الحالة ذات أقل كلفة فعلية من البداية.



Hill Climbing Search

المبدأ: أختار أقل تقدير $h(n)$ أي أختار الحالة التي تبدو الأقرب للهدف.



الآن نريد خوارزمية تختار العقدة التالية بحسب $g(n)$ و $h(n)$ معاً.

لتوازن بين الكلفة الفعلية والتقديرية وهي A^*

The A^* Algorithm (A-star)

■ نحسب لكل عقدة:

$$f(n) = g(n) + h(n)$$

■ $g(n)$ كلفة المسار من البداية إلى الحالة الحالية.

■ $h(n)$ تقدير الكلفة من الحالة الحالية إلى الحالة الهدف.

■ $f(n)$ المجموع الكلي أي التقدير التقريبي لتكلفة الحل الكامل لو سرنا عبر هذه العقدة.

ألية العمل:

■ نختار دائماً العقدة ذات أقل $f(n)$

لو كان heuristic من النوع admissible (يعني لاتبالغ) فإن:

- A^* تكون complete تجد الحل دائماً لو كان موجود.
- A^* تكون optimal تعطي أقصر طريق ممكن (الحل الأمثل)

ملاحظة:

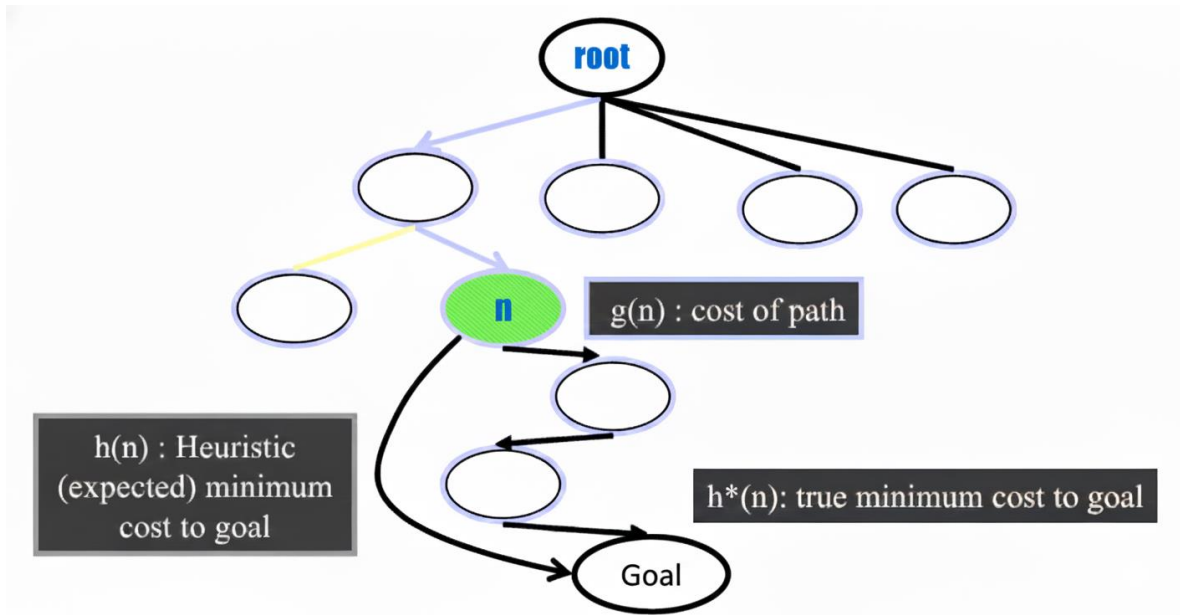
إذا $g(n)$ مأخوذ من خوارزمية الـ uniform cost والـ $h(n)$ مأخوذ من خوارزمية الـ hill climbing

عندما نقول heuristic admissible هذا يعني $h(n) \leq h^*(n)$

- يعني لا يوجد مبالغة أبداً في تقدير المسافة إلى الهدف حيث:
- $h(n)$: التقدير الذي نحسبه.
 - $h^*(n)$: الكلفة الحقيقية الفعلية من العقدة الحالية إلى الهدف.

ملاحظة:

في مثال Romania path عندما استخدمنا straight line فإن المسافة دائماً أصغر أو تساوي $\geq$ المسافة عبر الطرق لذلك اعتبرناه heuristic admissible



- $g(n)$: الكلفة من البداية إلى العقدة الحالية.
 - $h^*(n)$: الكلفة الحقيقية الفعلية من العقدة الحالية إلى الهدف.
 - $h(n)$: تقدير كلفة المسار من العقدة الحالية إلى الهدف.
 - $h(n) \leq h^*(n)$
- إذا كان التقدير optimistic أي لا يبالغ فإن A^* تضمن أن لا تتجاوز الحل الأمثل.

مثال تطبيق خوارزمية A* على خريطة رومانيا:

توضيح بالنسبة للأرقام:

الأرقام داخل المربعات هي $h(n)$ أما
الأرقام التي على الأسهم هي $g(n)$.

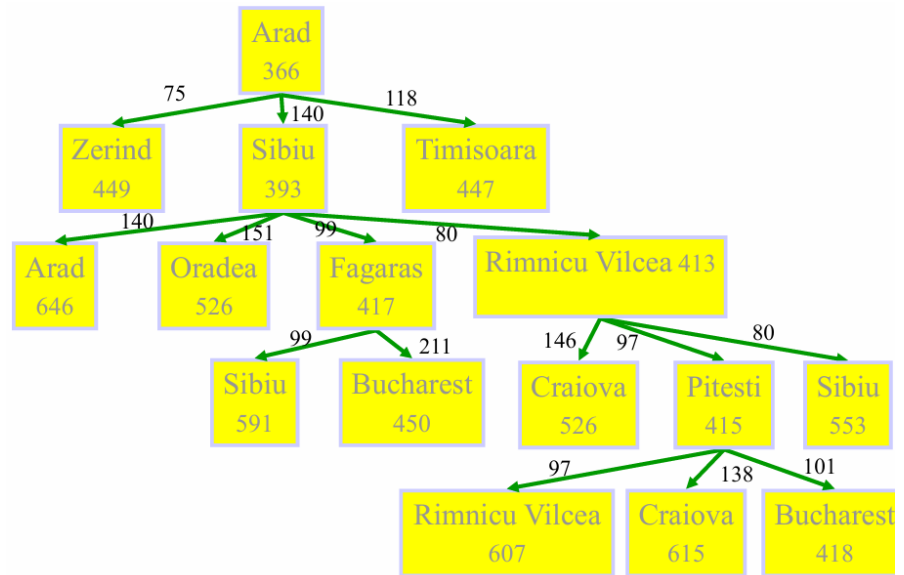
$$f(\text{zerind}) = g(\text{zerind}) + h(\text{zerind})$$

$$g(\text{zerind}) = g(\text{Arad}) + 75 = 0 + 75 = 75$$

$$f(\text{zerind}) = 75 + 449 = 524$$

وهكذا نتابع.....

$$f(\text{Arad}) = g + h = 366 + 0 = 366$$



خصائص خوارزمية A*:

complete -1

نعم طالما أن كل خطوة لها كلفة موجبة (لا يوجد صفر أو سالب)

optimal -2

نعم طالما الـ heuristic admissible

time -3

عالي لأنها قد تضطر لفتح العديد من العقد (Exponential).

space -4 $O(b^m)$

كبير لأنها تحتفظ بكل العقد المفتوحة.

❖ b : عامل التفرع branching factor

❖ m : العمق الأعظمي Maximum Depth

Informal proofing outline of A* :

البرهان غير الرسمي على اكتمال A* ومثالياتها:

complete -1

إذا كان لكل خطوة كلفة صغيرة موجبة δ
وهناك طريق محدود للوصول إلى الهدف فبعد عدد محدود من التوسعات سنصل إلى الهدف.
إذاً الخوارزمية لا تتوقف قبل إيجاد الحل.

optimal -2

عندما تجد A* الحل الأول الذي يكون لديه أقل $f(n)$ أي حل آخر ممكن سيكون له $f(n) \geq f(\text{goal})$
ولأن heuristic لا تبالغ فالحل الأول الذي نجده هو الأفضل.

مدى سرعة الـ A*:

A* هي أسرع خوارزمية بحث ممكنة طالما أننا نستخدم Heuristic أي لا يمكن لأي خوارزمية ان توسع عدداً أقل من
العقد مقارنة بـ A*

■ إذا كان الـ $h(n)$ مثالي (تطابق مع الكلفة الحقيقية)
A* تمشي إلى الهدف مباشرة بدون بحث.

■ إذا كان الـ $h(n)=0$ الخوارزمية تصبح uniform cost أي بطيئة.



انتصف طريق هذه الرحلة ...

الخوارزمية A* على شكل Pseudocode

```

1 OPEN = {InitialState}; CLOSE = nil; SUCCESS = False
2 While OPEN != {} And SUCCESS=False do
3   Begin
4     u = Get_First_Node_From_OPEN(OPEN)
5     OPEN = Queue(OPEN)
6     If GoalState(u) Then SUCCESS=True
7     Else
8       Begin
9         Add(u, CLOSE)
10        For each v in Children(u) do
11          Begin
12            If v Not in OPEN and v not in CLOSE Then
13              Add (v, OPEN)
14              Father(v)=u
15              Sort(OPEN)  % f ASC then g DECS
16            Else If v in OPEN and g(v)>g(u)+k(u,v) Then
17              g(v)=g(u)+k(u,v)
18              f(v)=g(v)+h(v)
19              Father(v)=u
20            Else If v in CLOSE and g(v)>g(u)+k(u,v) Then
21              g(v)=g(u)+k(u,v)
22              f(v)=g(v)+h(v)
23              Father(v)=u
24          End
25        End
26      End
27    If OPEN={} Then FAIL
28  Else
29    GenerateSolutionPath(u)

```



<See You Soon>